



哈爾濱工業大學

HARBIN INSTITUTE OF TECHNOLOGY



分布式存储与计算 (I)

一致性模型与复制

王宏志

哈尔滨工业大学

计算学部 海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

目录

Contents

- 1 复制与一致性基础
- 2 数据为中心的一致性模型
- 3 客户端为中心的一致性模型
- 3 一致性协议与复制实现

目录

Contents

- 1 复制与一致性基础
- 2 数据为中心的一致性模型
- 3 客户端为中心的一致性模型
- 3 一致性协议与复制实现

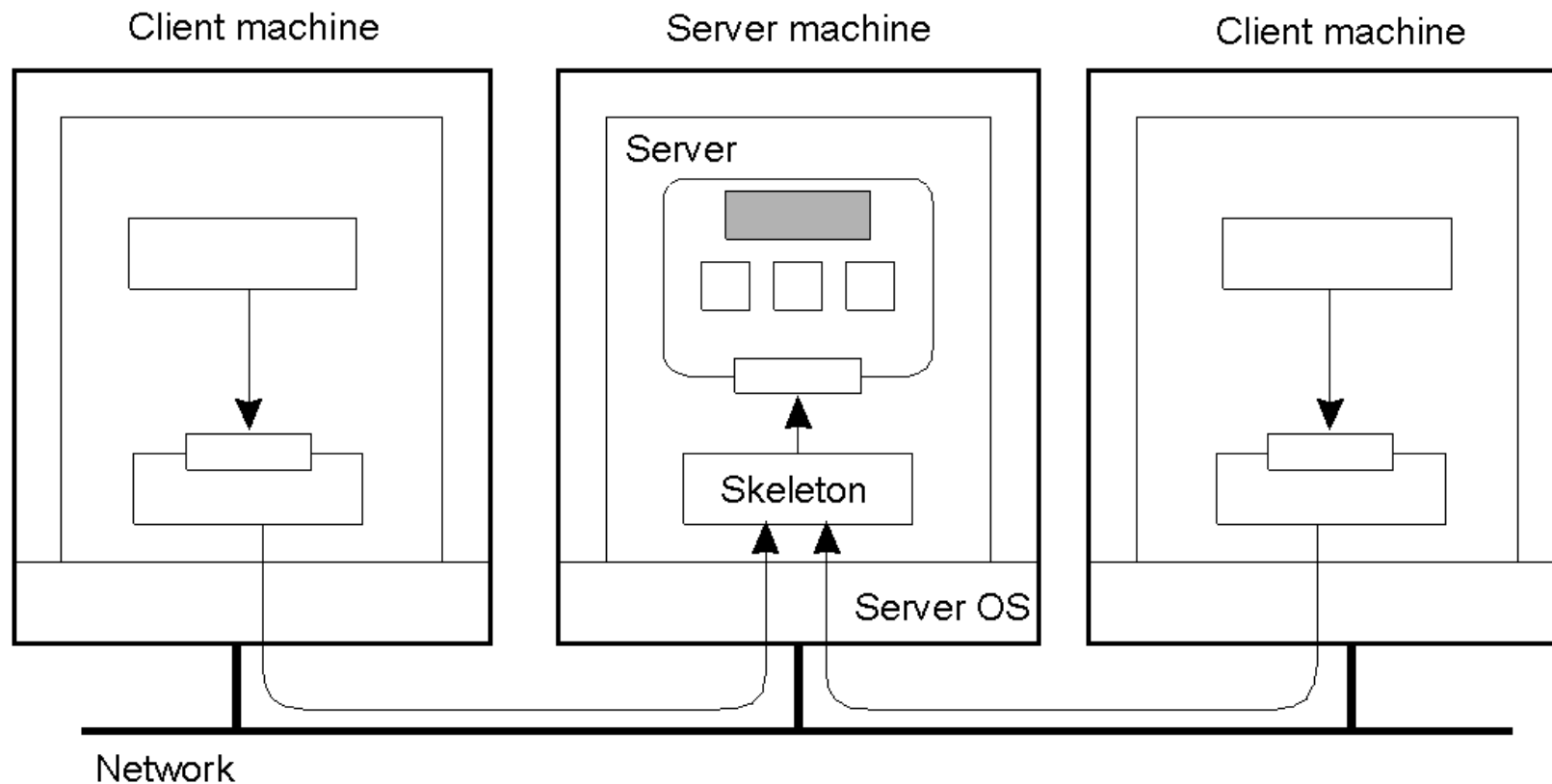
为什么要复制数据？

- 增强可靠性。
- 提高性能。
- 但是：如果有许多相同事物的副本，我们如何确保它们都保持最新？我们如何保持副本的一致性？
- 一致性可以通过多种方式实现。
- 我们将研究多种一致性模型，以及实现这些模型的协议。

更多关于复制

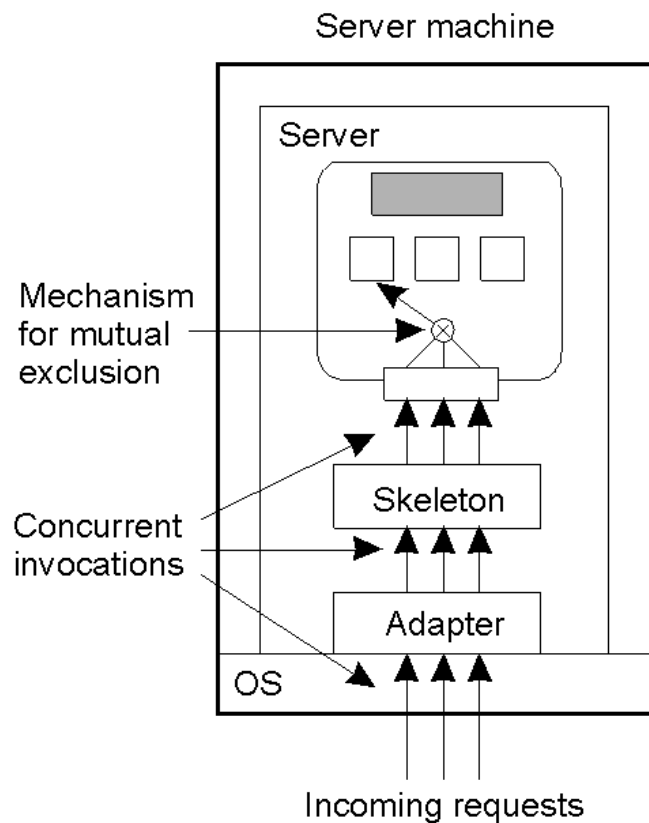
- Replicas 允许远程站点在本地发生故障时继续工作。
- 还可以防止数据损坏。
- 副本允许数据靠近其使用位置。
- 这直接支持分布式系统增强可扩展性的目标。
- 即使大量复制的"本地"系统也可以提高性能:想想集群。
- 那么,有什么问题呢?
- 要使所有这些复制品保持一致并不容易。

并发对象访问:问题

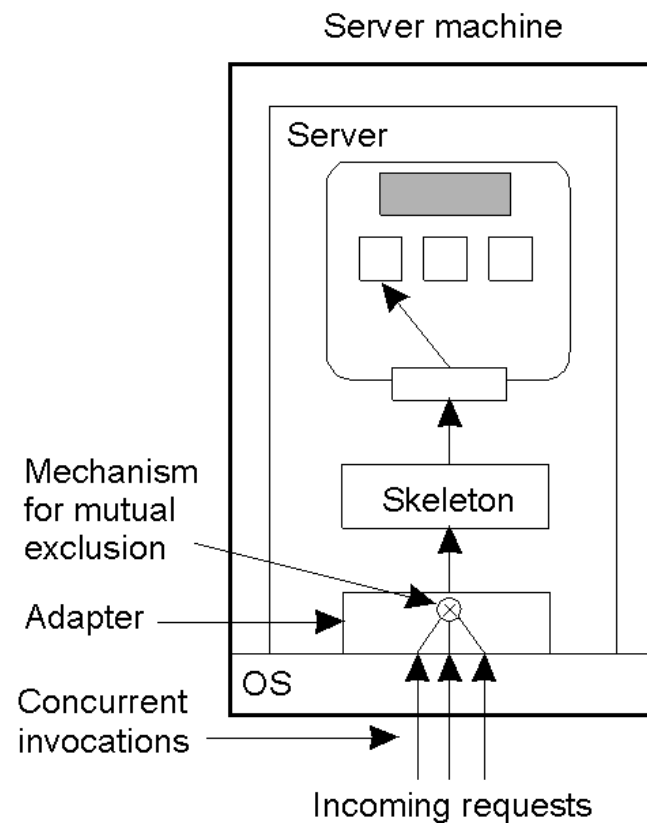


- 组织两个不同客户端共享的分布式远程对象。但是,在多个同时访问的情况下,如何保护对象呢?

并发对象访问:解决方案



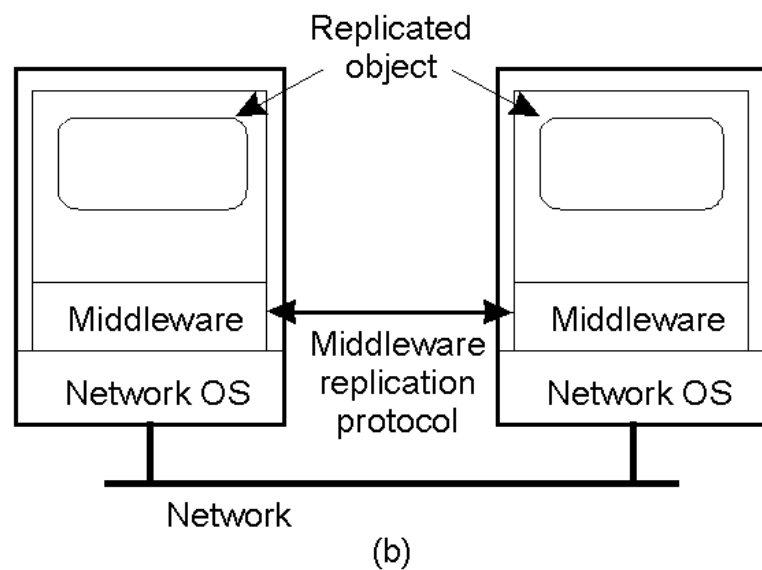
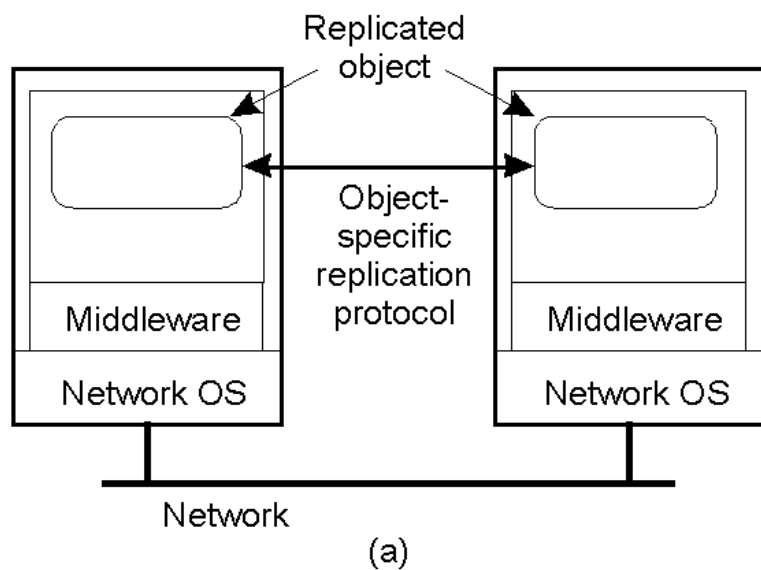
(a)



(b)

- a) 一个能够处理自身并发调用的远程对象。
- b) 需要对象适配器来处理并发调用的远程对象(依赖于中间件).

对象复制:解决方案



- a) 用于复制感知分布式对象的分布式系统 – 对象本身"知道"它被复制了.这是一个非常灵活的设置, 但可能会很昂贵, 因为DS开发者必须关注复制/一致性。
- b) 负责副本管理的分布式系统 – 灵活性较低,但减轻了 DS 开发人员的负担。这是最常见的方法。

复制和可扩展性

- 复制是一种广泛使用的可扩展性技术:想想 Web 客户端和 Web 代理。
- 当系统扩展时,**首先出现的问题**是与性能相关的问题——随着系统越来越大(例如,用户越来越多),它们往往变得更慢。
- 复制数据并将其**移近需要的地方**,有助于解决此可伸缩性问题。
- 还有一个问题:如何有效地同步所有创建的副本以解决可扩展性问题?
- **进退两难:添加副本可以提高可伸缩性,但会产生保持副本最新的开销(通常相当大)!!!**
- 如后文所见,其解往往放宽任何一致性约束。

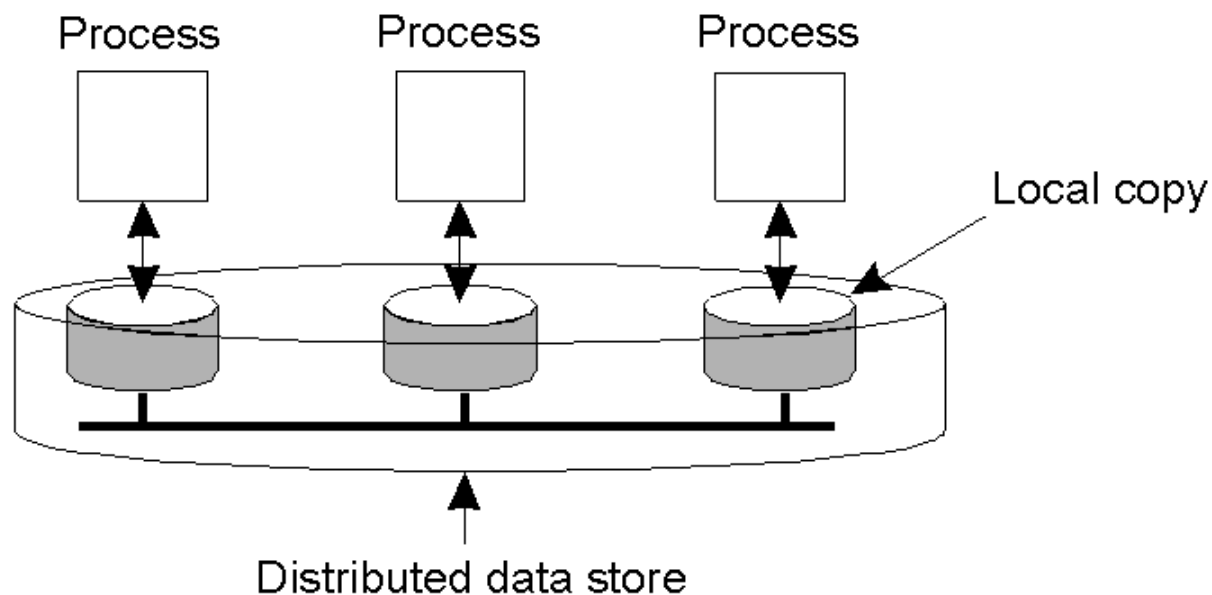
目录

Contents

- 1 复制与一致性基础
- 2 数据为中心的一致性模型**
- 3 客户端为中心的一致性模型
- 4 一致性协议与复制实现

以数据为中心的一致性模型

- 分布式系统中的任何进程都可以从数据存储中读取或写入数据存储。
- 数据存储(副本)的本地副本可以支持"快速读取"。
- 但是,对本地副本的写入需要传播到所有远程副本。



- Various consistency models help to understand the various mechanisms used to achieve and enable this.

什么是一致性模型?

- 一致性模型是DS数据存储与其进程之间的一个合约。
- 如果进程同意这些规则，数据存储将正常运行并按广告宣传的那样执行。
- 我们首先从严格一致性开始，它被定义为：
- *任何对数据项 'x' 的读取, 都会返回与最近对 'x' 的写入结果相对应的值(无论写入发生在何处)。*

一致性模型符号

- $W_i(x)a$ – 由进程 'i' 写入项目 'x' 的值为 'a'。也就是说, 'x' 被设置为 'a'。
- (注意: 进程通常表示为 'Pi')。
- $R_i(x)b$ – 由进程 'i' 从项目 'x' 读取产生值 'b'。也就是说, 读取 'x' 返回 'b'。
- 时间在所有图中从左到右移动。

严格一致性

P1: W(x)a

P2: R(x)a
(a)

P1: W(x)a

P2: R(x)NIL R(x)a
(b)

- 两个进程的行为,在同一数据项上运行:
 - a) 一个严格一致的数据存储。
 - b) 数据存储不严格一致。
- 使用 Strict Consistency,所有写入对所有进程都立即可见,并且在整个 DS 中保持绝对全局时间顺序。这就是一致性模型"圣杯"——在现实世界中一点也不容易,在 DS 中几乎不可能。
- 因此,又开发了其他不那么严格(或"较弱")的模型...

顺序一致性

- 较弱的一致性模型,这是规则的放松。
- 它也必须更容易(可能)实施。
- "顺序一致性"的定义:
- *任何执行的结果都是一样的,就像数据存储上所有进程的(读写)操作都以相同的顺序执行,并且每个进程的操作都按照其程序指定的顺序出现在此顺序中。*

顺序一致性

换句话说,所有进程都看到相同的交错操作集,而不管交错是什么。

P1:	W(x)a		
P2:		W(x)b	
P3:		R(x)b	R(x)a
P4:			R(x)b R(x)a

(a)

P1:	W(x)a		
P2:		W(x)b	
P3:		R(x)b	R(x)a
P4:			R(x)a R(x)b

(b)

- a) 顺序一致的数据存储 - 在所有副本上,"第一次"写入发生在"第二次"之后
- b) 不按顺序一致的数据存储 - 写入似乎以非顺序发生,这是不允许的。

顺序一致性的问题

- 有了这种一致性模型,调整协议以偏向读取而不是写入(反之亦然),可能会对性能产生毁灭性的影响。
- 为此,又提出了其他较弱的一致性模型。
- 同样,放宽规则,使这些较弱的模型变得有意义。

线性化与顺序一致性 (1)

Process P1	Process P2	Process P3
<pre>x = 1; print (y, z);</pre>	<pre>y = 1; print (x, z);</pre>	<pre>z = 1; print (x, y);</pre>

- 三个并发执行的进程。

线性化与顺序一致性 (2)

- 上一张幻灯片的四个有效执行顺序。 纵轴为时间

```
x = 1;  
print ((y, z);  
y = 1;  
print (x, z);  
z = 1;  
print (x, y);
```

Prints: 001011

Signature:
001011
(a)

```
x = 1;  
y = 1;  
print (x,z);  
print(y, z);  
z = 1;  
print (x, y);
```

Prints: 101011

Signature:
101011
(b)

```
y = 1;  
z = 1;  
print (x, y);  
print (x, z);  
x = 1;  
print (y, z);
```

Prints: 010111

Signature:
010111
(c)

```
y = 1;  
x = 1;  
z = 1;  
print (x, z);  
print (y, z);  
print (x, y);
```

Prints: 111111

Signature:
111111
(d)

但是,例如,001001是不允许的。

- 此模型区分了"因果关系"和不因果关系的事件。
- 若事乙为前事甲所致,或为前事甲所感,则因果之常,必令他事皆见甲,而后见事乙。
- 非因果关系的操作称为并发。

更多关于因果一致性

- 因果一致的数据存储服从此条件:
- 可能因果相关的写入,必须由所有进程以相同的顺序看到。 并发写入在不同的机器上可能以不同的顺序(即不同的进程)看到。

P1:	W(x)a		W(x)c	
P2:		R(x)a	W(x)b	
P3:		R(x)a		R(x)c
P4:		R(x)a		R(x)b

此数在因果一致的存储中是允许的,而对于顺序或严格一致的存储则不允许。 注:假设W2(x)b与W1(x)c并发。

另一个因果一致性示例

P1:	W(x)a		
P2:		R(x)a	W(x)b
P3:			R(x)b R(x)a
P4:			R(x)a R(x)b
(a) incorrect			

P1:	W(x)a		
P2:		W(x)b	
P3:			R(x)b R(x)a
P4:			R(x)a R(x)b
(b) correct			

- a) 违反因果一致性– P2 的写与 P1 的写相关,因为读 'x' 给出 'a'(所有进程必须以相同的顺序看到它们).
- b) 因果一致的数据存储:读取已被删除,因此两个写入现在是并发的。 P3 和 P4 的读数现在可以了。

先进先出一致性

- 定义如下:

单个进程所写,所有其他进程都按其发出的顺序看到,但不同进程的写入,不同进程可能以不同的顺序看到.

- 这也称为"PRAM 一致性"——流水线 RAM。
- 先进先出的魅力在于易于实施。 不能保证不同进程看到写入的顺序,但必须按顺序看到来自一个进程的两个或多个写入。

FIFO 一致性示例

P1:	W(x)a			
P2:	R(x)a	W(x)b	W(x)c	
P3:			R(x)b	R(x)a R(x)c
P4:			R(x)a	R(x)b R(x)c

- FIFO 一致性事件的有效序列。
- 注意,迄今为止所研究的一致性模型都不允许这种事件序列。

引入弱一致性

- 并非所有应用程序都需要查看所有写入,更不用说以相同的顺序查看它们了。
- 这导致了"弱一致性"(主要用于处理分布式关键区域)。
- 该模型引入了"同步变量"的概念,用于更新数据存储的所有副本。

弱一致性特性

- 弱一致性的三个属性：
 1. 对与数据存储相关的同步变量的访问是按**顺序一致**的。
 2. 在**所有先前的写入都完成之前**,不允许对同步变量进行任何操作。
 3. 在对同步变量的所有先前操作都执行完毕之前,不允许对数据项执行读取或写入操作。

弱一致性:其意义

- 所以.....
- 通过执行同步,进程可以将刚刚写入的值强制到所有其他副本中。
- 此外,通过同步,进程可以确保在读取之前获得最近写入的值。
- 本质上,弱一致性模型对一组操作强制保持一致性,而不是单个读写一致性(如严格一致性、顺序一致性、因果一致性和FIFO 一致性)。

弱一致性的例子

P1:	W(x)a	W(x)b	S
P2:		R(x)a	R(x)b S
P3:		R(x)b	R(x)a S

(a)

在同步之前,任何结果都可以接受

P1:	W(x)a	W(x)b	S
P2:		S	R(x)a

(b)

错了!

- a) 弱一致性的有效事件序列。这是因为 P2 和 P3 尚未同步,因此无法保证 'x' 中的值。
- b) 弱一致性的无效序列。P2 已同步,因此不能从 "x" 读取 "a",它应该得到 "b" 。

- 问题：一个弱一致性数据存储如何知道同步是读取还是写入的结果？
- 答案：它不知道！
- 如果数据存储能够确定同步是读取还是写入，则可以实施效率。
- 可以使用两个同步变量，“获取”和“释放”，它们的使用导致了“释放一致性”模型。

- 定义如下:

当一个进程执行“获取”操作时, 数据存储将确保所有受保护数据的本地副本在必要时更新, 以与远程副本保持一致。

当执行“释放”操作时, 已更改的受保护数据将传播到数据存储的本地副本。

发布一致性示例

P1:	Acq(L)	W(x)a	W(x)b	Rel(L)	
P2:			Acq(L)	R(x)b	Rel(L)
P3:					R(x)a

- 释放一致性的有效事件序列。
- 过程 P3 未执行获取,因此不能保证读取 'x' 一致。 数据存储只是没有义务提供正确答案。
- P2 确实执行了获取,因此其读取的"x"是一致的。

发布一致性规则

- 分布式数据存储如果遵守以下规则,则为"发布一致":
 1. 在对共享数据执行读写操作之前,该进程之前所做的所有获取都必须成功完成。
 2. 在允许执行发布之前,进程之前的所有读取和写入都必须完成。
 3. 对同步变量的访问是 FIFO 一致的(不需要顺序一致性)。

引入条目一致性

- 另一种变化是"入口一致性"。 仍使用获取和发布,并且数据存储满足以下条件:
 1. 在对受保护的共享数据执行所有更新之前,不允许对该进程执行同步变量的获取访问。
 2. 在允许进程对同步变量进行独占模式访问之前,其他进程不得持有该同步变量,即使在非独占模式下也是如此。
 3. 在对同步变量执行了独占模式访问之后,任何其他进程对该同步变量的下一个非独占模式访问,在对该变量的所有者执行之前,不得执行。

条目一致性:其含义

- 因此,在采集时,所有对受保护数据的远程更改都必须是最新的。
- 在写入数据项之前,进程必须确保没有其他进程同时尝试写入

P1:	Acq(Lx)	W(x)a	Acq(Ly)	W(y)b	Rel(Lx)	Rel(Ly)
P2:			Acq(Lx)	R(x)a		R(y)NIL
P3:				Acq(Ly)		R(y)b

锁与单个数据项相关联,而不是与整个数据存储相关联。

注意:P2 读取 'y' 返回 NIL,因为没有请求锁。

一致性模型摘要

- a) 不使用同步操作的一致性模型。
- b) 使用同步操作的模型。（这些需要额外的编程结构,并允许程序员将数据存储视为顺序一致,而实际上并非如此。他们"应该"也提供最好的性能)。

Consistency	Description
Strict	Absolute time ordering of all shared accesses matters.
Linearizability	All processes must see all shared accesses in the same order. Accesses are furthermore ordered according to a (nonunique) global timestamp.
Sequential	All processes see all shared accesses in the same order. Accesses are not ordered in time.
Causal	All processes see causally-related shared accesses in the same order.
FIFO	All processes see writes from each other in the order they were used. Writes from different processes may not always be seen in that order.

(a)

Consistency	Description
Weak	Shared data can be counted on to be consistent only after a synchronization is done.
Release	Shared data are made consistent when a critical region is exited.
Entry	Shared data pertaining to a critical region are made consistent when a critical region is entered.

(b)

目录

Contents

- 1 复制与一致性基础
- 2 数据为中心的一致性模型
- 3 客户端为中心的一致性模型**
- 4 一致性协议与复制实现

前所研究的一致性模型,在于在并发读写操作的情况下,保持一致的(全局可访问的)数据存储。另一类分布式数据存储,其特点是没有同时更新。在这里,重点在于为当前在数据存储上运行的单个客户端进程保持一致的事物视图。

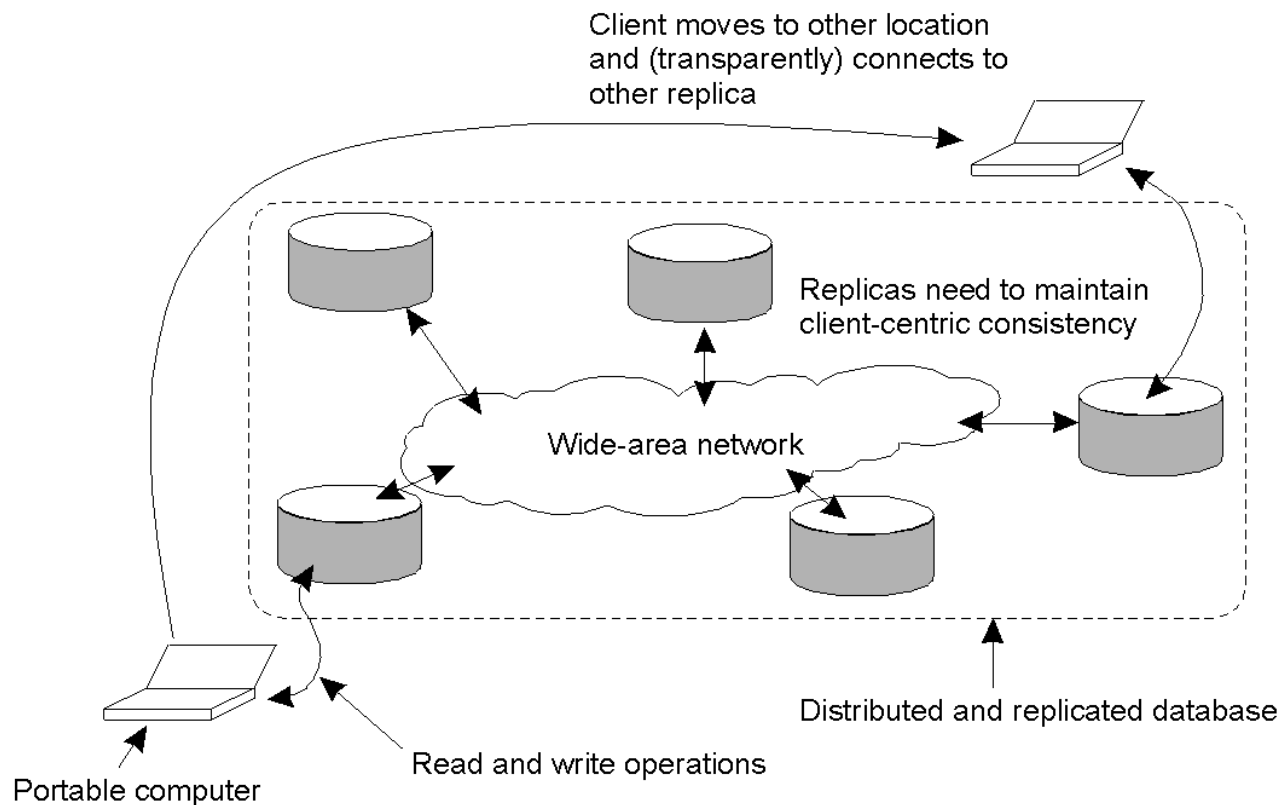
更以客户为中心的一致性

- 只读进程的更新(写入)速度应该有多快?
- 想想大多数数据库系统:主要是`read`.
- 想想 DNS:不会发生写-写冲突.
- 想想 WWW:与 DNS 一样,只是大量使用客户端缓存:即使返回陈旧的页面,大多数用户都可以接受.
- 这些系统都表现出高度可接受的不一致..... 随着时间的推移,复制品逐渐保持一致。

走向最终的一致性

- 唯一的要求是所有复制品最终都是一样的。
- 所有更新都必须保证传播到所有副本…… 最终!
- 如果每个客户端总是更新相同的副本,则效果很好。
- 如果客户是移动的,事情就有点困难了。

最终一致性:移动问题

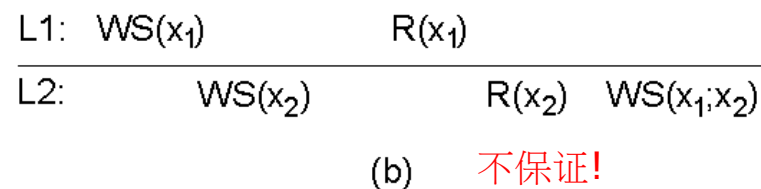
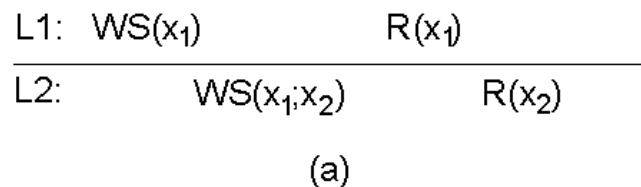


- 移动用户访问分布式数据库的不同副本的原理。
- 当系统能够保证单个客户端以一致的方式看到对数据存储的访问时,我们就说"以客户端为中心的一致性"成立。

一个例子:Bayou系统

- Bayou System 实施了 4 种以客户为中心的一致性模型:
- 单调读一致性
- 单调写一致性
- 读写一致性
- 写后读一致性

- 单调读操作: 如果进程读取数据项 'x' 的值, 则该进程对 'x' 进行任何连续读取操作, 将始终返回相同的值或较新的值.



- 单个进程 P 在同一数据存储的两个不同本地副本上执行的读取操作.
 - a) 单调读取一致的数据存储
 - b) 不提供单调读取的数据存储。

- 单调写入: 进程对数据项 'x' 的写操作, 在同一进程对 'x' 的连续写操作之前完成。

L1: W(x₁)

L2: W(x₁) W(x₂)

(a)

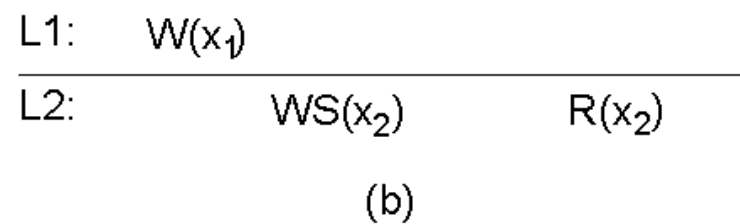
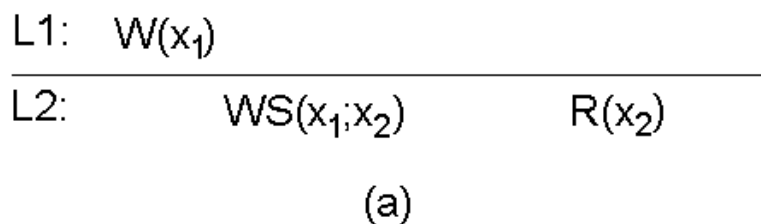
L1: W(x₁)

L2: W(x₂)

(b)

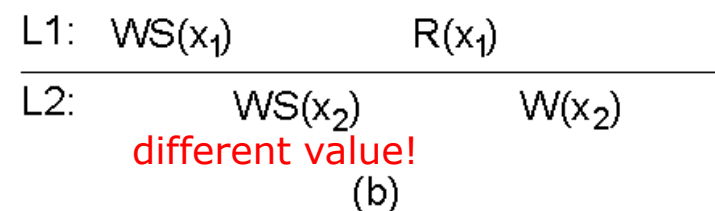
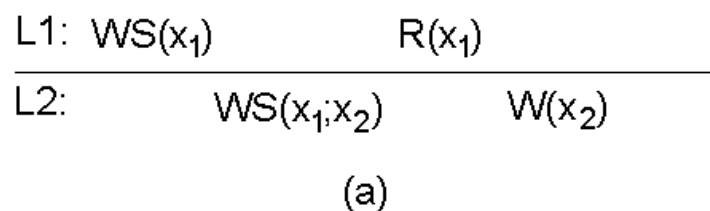
- 单个进程 P 在同一数据存储的两个不同本地副本上执行的写入操作
- 单调写入一致的数据存储。
- 不提供单调写入一致性的数据存储。

- 读取写入：一个进程对数据项 'x' 的写入操作的效果，总会在同一进程对 'x' 的后续读取操作中看到。



- a) 提供读写一致性的数据存储。
- b) 一个没有的数据存储。

- 写入后读取：一个进程在数据项 'x' 上执行写入操作，紧随之前该进程对 'x' 的读取操作，保证在读取的同一或更晚的 'x' 值上发生。



- a) 写后读一致数据存储
- b) 不提供写后读一致性的数据存储

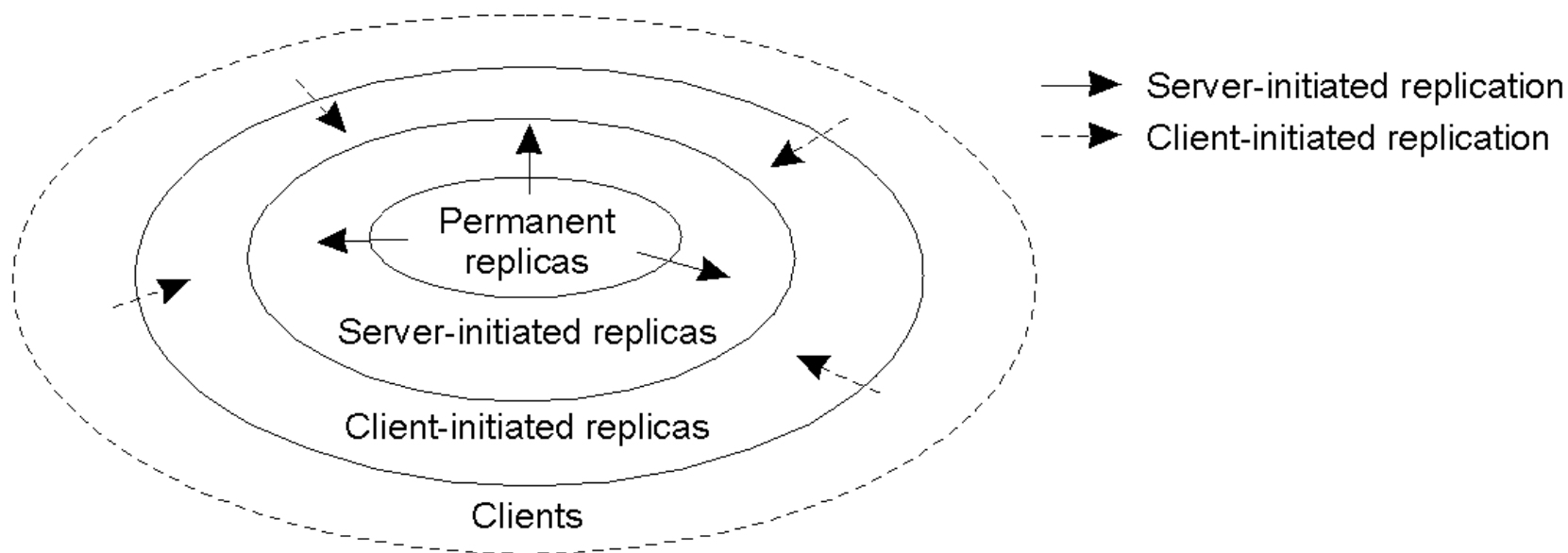
目录

Contents

- 1 复制与一致性基础
- 2 数据为中心的一致性模型
- 3 客户端为中心的一致性模型
- 4 一致性协议与复制实现**

分发协议

- 无论选择哪种一致性模型,我们都需要决定在何处、何时以及由谁放置数据存储的副本。



副本放置类型

- 有三种类型的副本：
- **永久副本**：通常数量较少，组织成COW（工作站集群）或镜像系统。
- **由服务器启动的副本**：用于在数据存储所有者启动时提高性能。通常由网络托管公司使用，将副本地理位置靠近最需要的地方。（通常被称为“推送缓存”）。
- **由客户端启动的副本**：作为客户端请求的结果创建——想想浏览器缓存。当然，假设缓存的数据不会很快过时，效果很好。

- 当客户端启动对分布式数据存储的更新时,传播什么?
- 有三种可能:
 1. 将更新通知传播到其他副本 – 这是一种"失效协议",表示副本的数据不再是最新的。写入多时效果很好.
 2. 将数据从一个副本传输到另一个副本 – 当读取次数多时效果很好.
 3. 将更新传播到其他副本 – 这是"活动复制",并在"初始写入"时将工作负载转移到每个副本。

- 另一个设计问题与更新是推送还是拉取有关?
 1. **推送式/ 服务器式方法**: 由服务器"自动"发送, 客户端不请求更新。 当需要高度一致性时, 这种方法很有用。常用于永久副本和服务器启动副本之间。
 2. **基于拉取/ 基于客户端的方法**: 客户端缓存（例如，浏览器）使用，客户端从服务器请求更新。无请求，无更新！

推送与拉取协议：权衡

问题	基于推送	基于拉取
服务器上的状态。	客户端副本和缓存的列表。	没有。
发送的消息。	更新(并可能稍后获取更新)。	拉取并更新。
客户响应时间。	立即(或获取更新时间)。	抓取更新时间。

- 在多客户端、单服务器系统的情况下,基于推送的协议和基于拉取的协议的比较。
- 混合方案是可能的:例如,"租赁"——服务器承诺在一段时间内将更新推送到客户端。租期满后,客户端将恢复拉取方法(直到再签发租约为止)。

- 这是一类有趣的协议,可用于实现 **Eventual Consistency**(注:这些协议在 Bayou 中使用)。
- 主要关注的是以尽可能少的消息数传播到所有副本的更新。
- 当然,我们在这里传播的是更新,而不是疾病!
- 有了这个"更新传播模型",其目的是尽快"感染"尽可能多的副本。

- 服务器类型:
- **感染性副本**:保存可传播到其他副本的更新的服务器.
- **易感副本**:一个尚未更新的服务器.
- **删除的副本**:不会(或不能)将更新传播到任何其他副本的更新服务器.
- 诀窍是尽快使所有易感服务器处于感染或删除状态,而不遗漏任何副本.

反熵协议

- 熵:"宇宙退化或紊乱的量度"。
- 服务器 P 随机选择 Q 并交换更新,使用三种方法之一:
 1. P 只推到 Q。
 2. P 只从 Q 中提取。
 3. P和Q互相推拉。

迟早,系统中的所有服务器都会被感染(更新)。 效果很好。

The Gossiping Protocol

- 这种变体被称为"流言蜚语"或"谣言传播",其作用如下:
 1. P 刚刚更新了 'x' 项。
 2. 它立即将"x"的更新推送到 Q。
 3. 如果 Q 已经知道"x",则 P 不愿再传播任何更新(谣言)而被删除。
 4. 否则 P 会向另一个服务器闲言碎语,Q 也是如此。
- 这种方法很好,但可以证明不能保证所有更新都传播到所有服务器。

- 反熵和流言蜚语的混合被认为是用更新快速感染系统的最好方法.
- 然而,删除数据呢?
- 更新容易,删除难得多!
- 在某些情况下,删除后,在某个副本上可能会出现对已删除项目的"旧"引用,并导致删除的项目被重新激活!
- 一种解决办法是为数据项目签发"死亡证明"——这是一种特殊的更新方式。
- 唯一剩下的问题是最终删除"旧"死亡证明(超时可以帮助)。

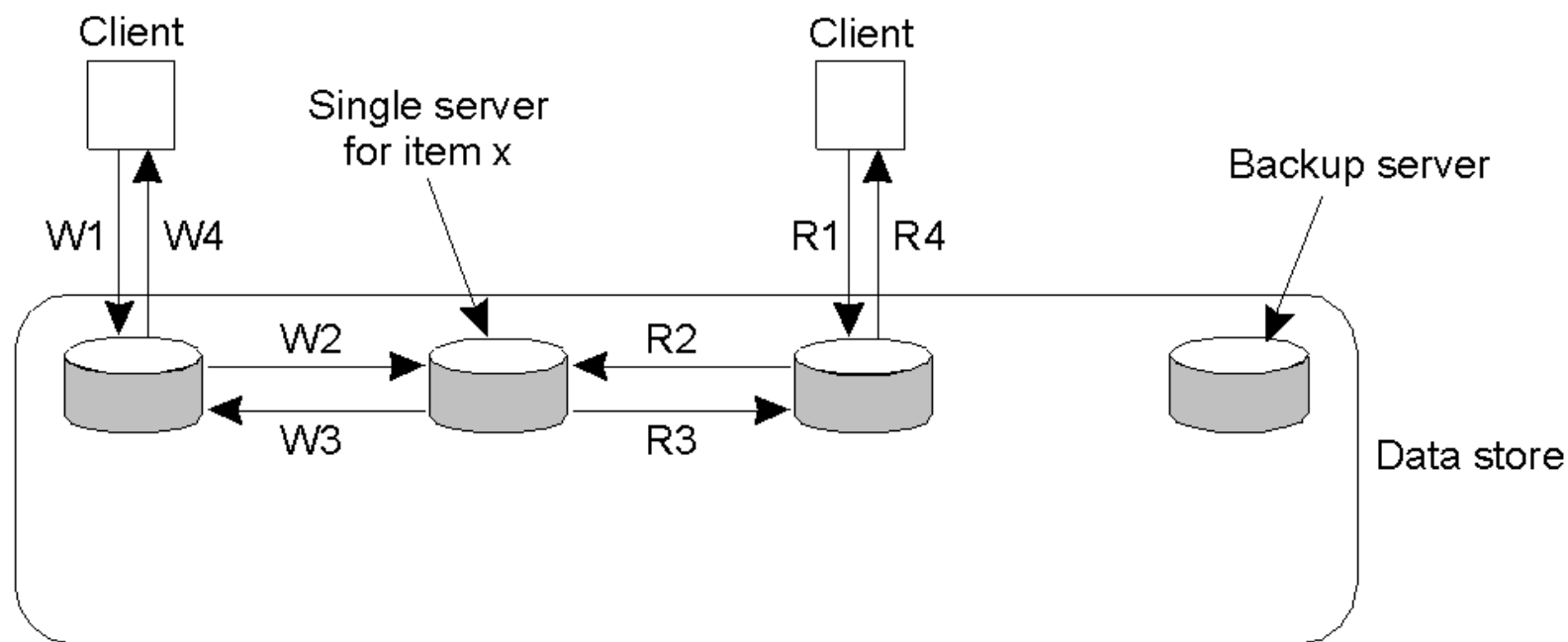
- 这是一致性模型的具体实现。
- 最广泛实施的模式是：
 1. 顺序一致性。
 2. 弱一致性(有同步变量)。
 3. 原子交易(即将研究)。

Primary-Based Protocols

- 每个数据项都与一个"主"副本相关联。
- 主服务器负责协调对数据项的写入。
- 基于主的协议有两种类型：
 1. 远程写入。
 2. 本地写入。

远程写入协议

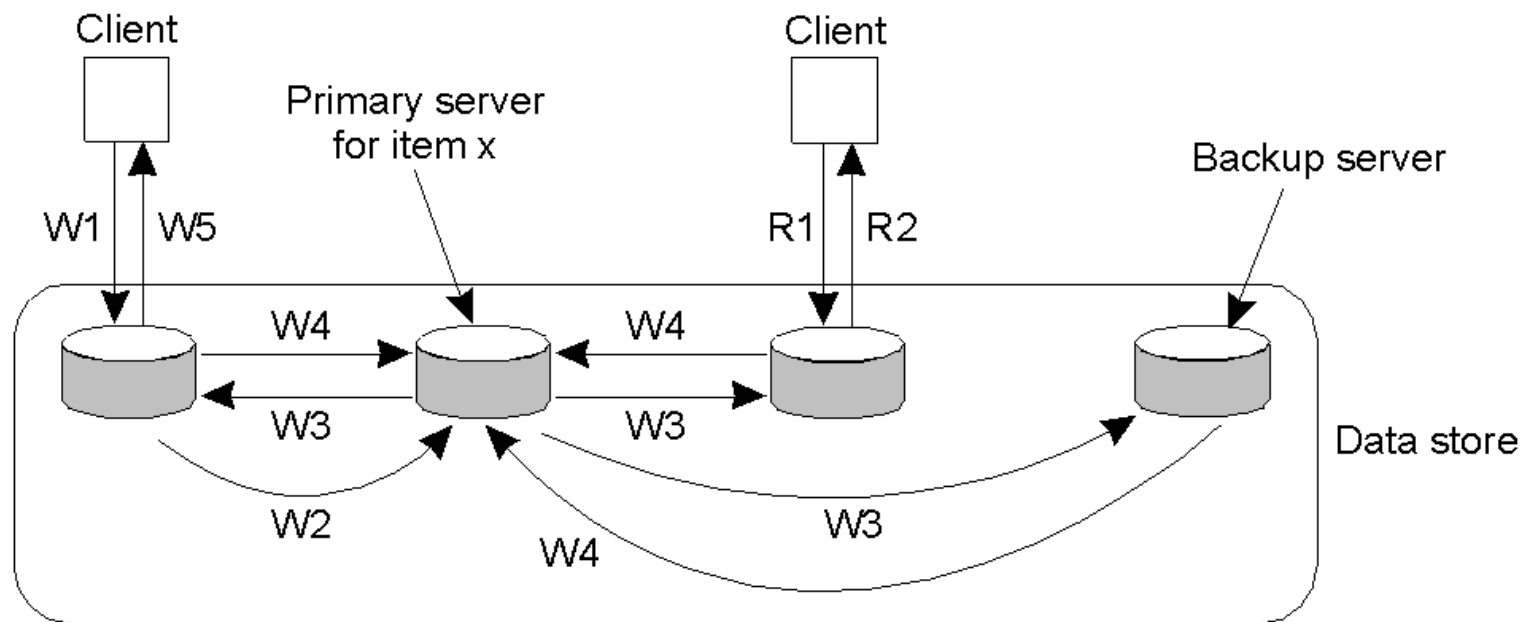
使用此协议,所有写入都在单个(远程)服务器上执行。
此模型通常与传统的客户端/服务器系统相关联。



W1. Write request
W2. Forward request to server for x
W3. Acknowledge write completed
W4. Acknowledge write completed

- R1. Read request
- R2. Forward request to server for x
- R3. Return response
- R4. Return response

主备份协议:一种变体



W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

写入仍然是集中的,但读取现在是分布式的。主坐标写入每个备份。

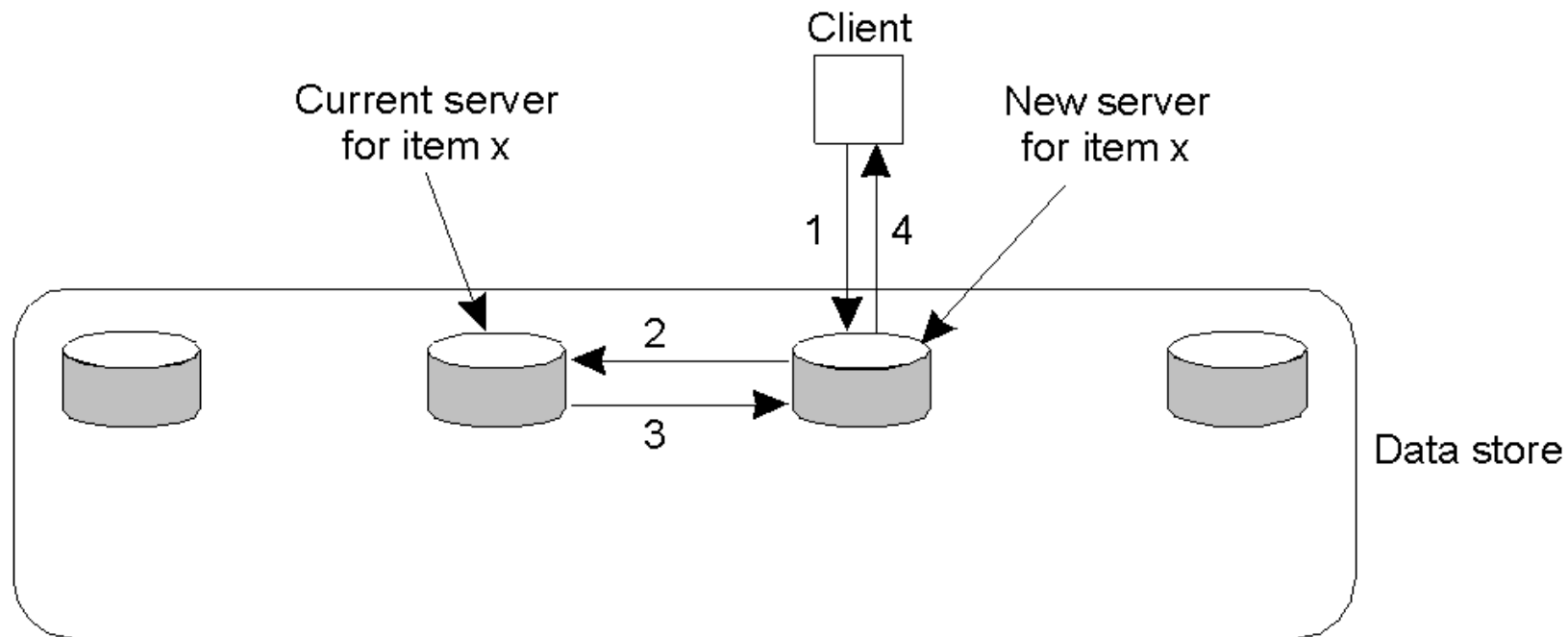
主备份的坏处和好处

- 坏处:性能!
- 所有这些写入都需要很长时间(尤其是当使用"阻塞写入协议"时)。
- 使用非阻塞写入协议来处理更新可能会导致容错问题(这是我们的下一个主题)。
- 好处:此方案的好处是,由于主服务器在控制之中,所有写入都可以以相同的顺序发送到每个备份副本,从而易于实现顺序一致性。

本地写入协议

- 在此协议中,仍保留数据项的单个副本。
- 写入时,数据项被转移到正在写入的副本中。
- 也就是说,数据项的主状态是可转移的。
- 这也称为"完全迁移方法"。

本地写入协议示例



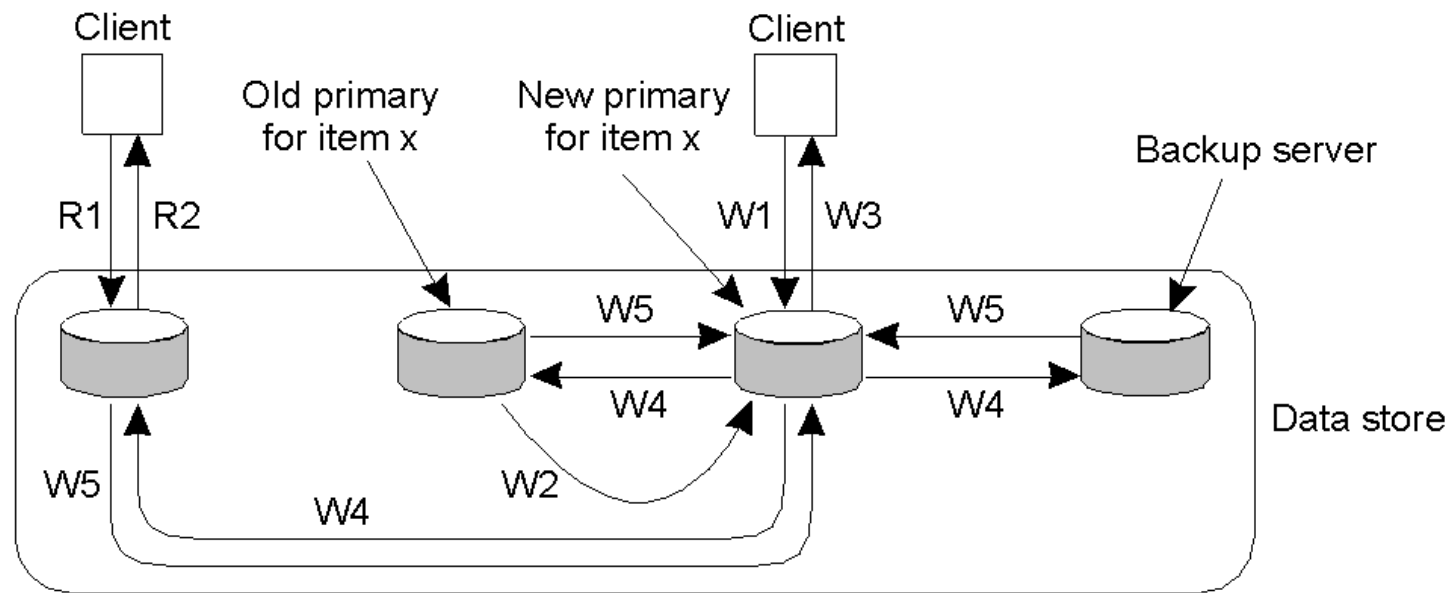
1. Read or write request
2. Forward request to current server for x
3. Move item x to client's server
4. Return result of operation on client's server

- 基于主的本地写入协议,在进程之间迁移单个副本(在读/写之前)。

本地写入问题

- 任何将要读取或写入数据项的进程所要回答的大问题是：
- "数据项现在在哪里?"
- 可以使用本课程前面所学的一些动态命名技术,但缩放很快就成了问题。
- 进程实际定位数据项所花费的时间可能比使用数据项花费的时间更多!

本地写入协议:一种变体



W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

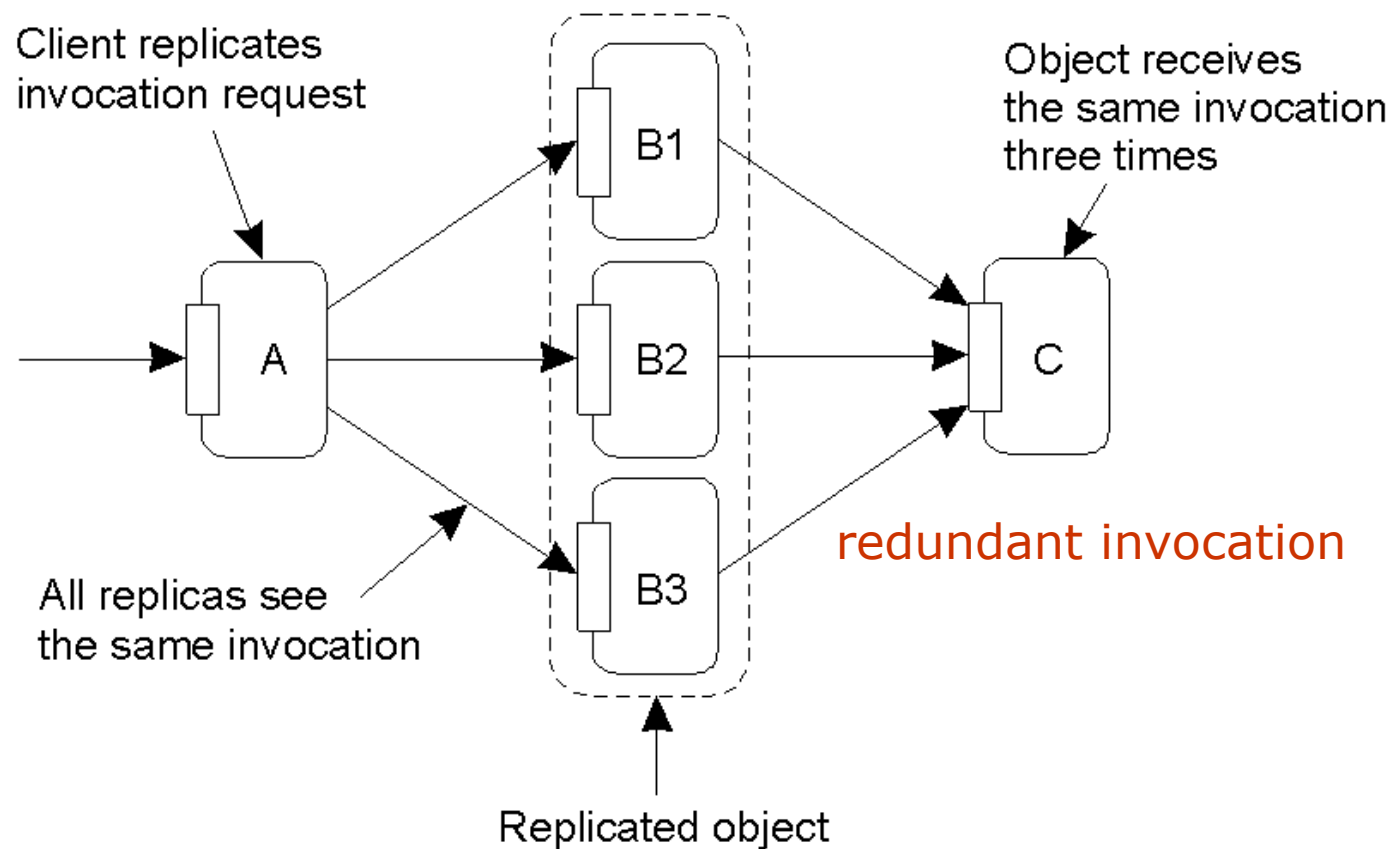
- 主备份协议,其中主迁移到要执行更新的进程,然后更新备份。 因此,读取效率要高得多。

复制写协议

- 有了这些协议,就可以在任何副本上进行写入。
- 另一个名称可能是:"分布式写入协议"
- 有二种:
 1. 主动复制。
 2. 多数票 (法定人数) 。

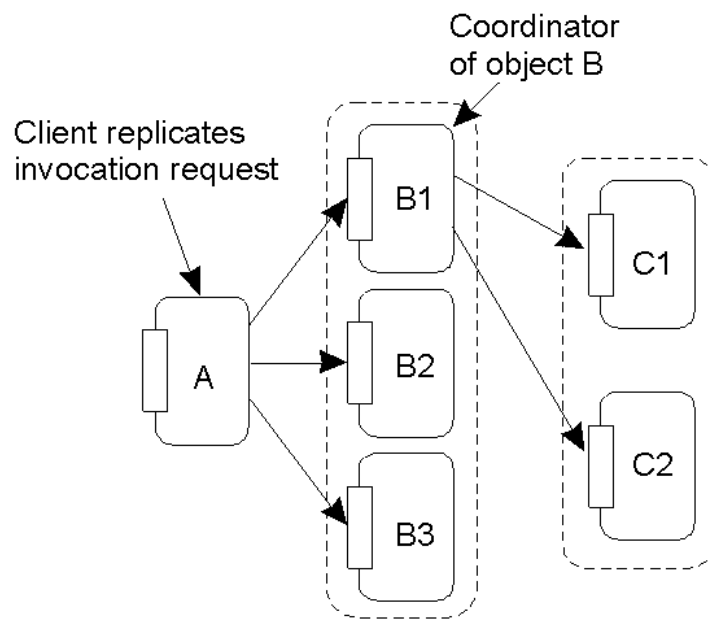
- 一个特殊的过程在每个副本上执行更新操作。
- Lamport 的 timestamps 可用于实现总排序,但这在分布式系统中不能很好地扩展。
- 另一种/变体是使用排序器,这是一个为每次更新分配一个**唯一 ID#** 的过程,然后传播到所有副本。
- 这可能导致另一个问题:**重复调用**。

主动复制:问题

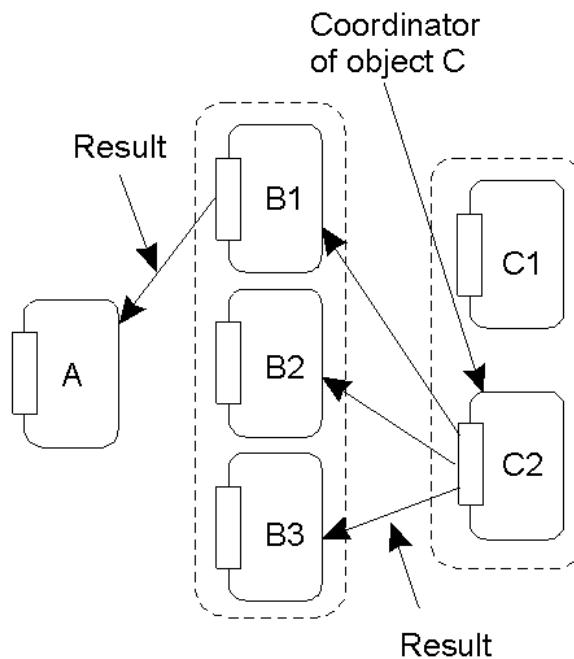


- 复制调用的问题——"B"是复制对象(它本身称为"C")。当"A"调用"B"时, 如何确保"C"不被调用三次?

主动复制:解决方案



(a)



(b)

- a) 对 'B' 使用协调器,它负责将调用请求从复制对象转发到 'C' 。
- b) 从"C"返回结果,其原理相同:协调者负责将结果返回给所有"B"。 注意返回 "A"的单个结果。

基于法定人数的协议

- 客户端必须先从多个副本请求并获取权限,然后才能读取/写入复制的数据项。

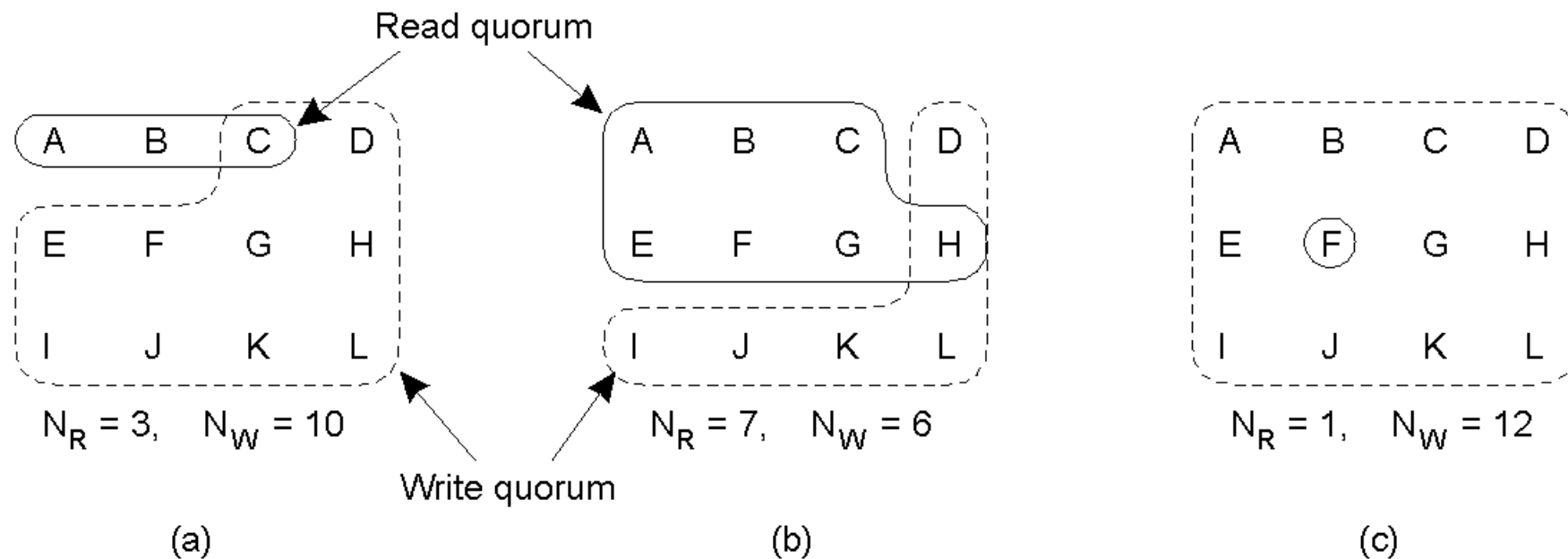
试看这个例子:

- 文件在分布式文件系统中复制。
- **要更新文件**,进程必须得到大多数副本的批准才能执行写入。副本必须同意同时执行写入。
- 更新后,文件有一个新版本 # 与之关联(并在所有更新的副本上设置)。
- **要读取**,进程会联系大多数副本,并要求提供文件的版本 #。如果版本 # 相同,则该文件必须是最新版本,然后可以继续读取。

- $N_R + N_W > N$

- $N_W > N/2$

基于法定人数的协议



- 投票算法的三个例子:
- a) 正确选择读写集
- b) 一个可能导致写作冲突的选择
- c) 正确的选择,称为 ROWA(读-写全)

缓存相干协议

- 这些都是特例,因为缓存通常由客户端而不是服务器控制。
- 相干检测策略:
- 何时发现不一致?
- 编译时静态:插入额外指令。
- 在运行时动态:与服务器检查的代码。
- 连贯执行策略
- 如何保持缓存的一致性?
- 服务器发送:失效消息。
- 更新传播技术, 组合是可能的。

写入缓存呢?

- **只读缓存**: 更新由服务器(即推送)或由客户端(即客户端发现缓存陈旧时拉取)执行。
- **直写缓存**: 客户端修改缓存,然后将更新发送到服务器。
- **回写缓存**: 延迟更新的传播,允许在本地进行多次更新,然后将最新的更新发送到服务器(这可以对性能产生巨大的积极影响)。

一致性与复制:摘要

- 复制的原因:性能提高,可靠性提高。
- 复制会导致不一致.....
- 如何才能最好地传播更新,使这些不一致之处不被注意到?
- "最好"的意思是"没有残废的性能"。
- 所提出的解决方案围绕放宽任何现有的一致性约束来解决。

- 人们提出了各种一致性模型：
- 严格、顺序、因果、FIFO 关注对数据项的单个读写。
- 较弱的模型引入了同步变量的概念：释放、进入与一组读写有关。
- 这些模型被称为"以数据为中心"。
- "以客户为中心"的模式也存在：
- 关注保持单个客户端访问分布式数据存储的一致性。
- 最终一致性模型就是一个例子。

- 为了分发(或"传播")更新,我们区分了传播的内容、传播的地方和由 WHO 传播。
- 我们查看了各种分发协议和一致性协议,旨在促进更新的传播。
- 最广泛实施的方案是那些支持顺序一致性或与同步变量的弱一致性的方案。